

UMBC Retriever Essentials Dashboard

Software Requirements Specification (SRS)

Version 1.0

09/28/2026

Hader Hamayun, Mosi Farquharson, Raian Mamun, Royce

Nogales

Table of Contents

1. Introduction

- 1.1. Purpose
- 1.2. Scope
- 1.3. Intended Audience
- 1.4. Definitions
- 1.5. References

2. Overall Description

- 2.1. Description
- 2.2. User Classes
- 2.3. Operating environment
- 2.4. Design and Implementation Constraints
- 2.5. Assumptions
- 2.6. Dependencies

3. Business Rules

4. Functional Requirements

- 4.1. Department Contributors (Staff)
- 4.2. Leadership (Dean of Students, Chief of Staff)
- 4.3. System Administrators

5. Use Cases

- 5.1. Use Case 1
- 5.2. Use Case 2
- 5.3. Use Case 3
- 5.4. Use Case 4
- 5.5. Use Case 5

6. Data Requirements

6.1. Data Requirements

6.2. Data Dictionary

6.3. Data Maintenance

7. External Interface Requirements

7.1. User Interface

7.2. Software Interface

7.3. Communication Interface

8. Nonfunctional Requirements

8.1. Availability Requirements

8.2. Reliability Requirements

8.3. Scalability Requirements

8.4. Security Requirements

9. Reporting Requirements

9.1. Report 1

10. Supplemental Info

10.1. Frontend

10.2. Backend

10.3. Coding Practices

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification(SRS) document is to define and outline the functional requirements for the initial version (1.0) of the UMBC Retriever Essentials Dashboard. This document is used to serve as the foundation for the development of this project as well as a reference for the stakeholders involved in this project. All requirements specified in this document are expected to be fully completed by the end of the semester.

1.2. Scope

The existing problem that various organizations at UMBC have is a lack of a central hub where they can see overall quantitative trends of the students enrolled for the organizations that are built to help students in their journey in academia at this school. These tools are not available per the software each of these organizations use and are unable to combine the data to view different visual trends with aggregate data. Therefore, the Retriever Essentials Dashboard is intended to be a visual representation of these trends that multiple organizations are able to use without having to depend on the capabilities of the current software they are using.

1.3. Intended Audience

The intended audience of this document is specifically aimed at the members of the development team, the project manager overseeing the development team, and the stakeholders of this project. The development team is composed of Hader Hamayun, Royce Nogales, Raian Mamun, and Mosi Farquharson. The project management team consists of Dr. Samit Shivadekar and his supporting staff with Dr. Shivadekar being the main subject matter expert in the development of this project. Finally, the project stakeholders consist of Micheal Berardi as well as the main Administration of UMBC. Micheal Berardi will be the main stakeholder to contact throughout the project.

1.4. Definitions

- 1.4.1. Subject Matter expert: someone who has specialized knowledge in a particular area of work or activity

Cambridge University Press & Assessment. (n.d.). Subject matter expert. In Cambridge dictionary. Retrieved September 28, 2026, from <https://dictionary.cambridge.org/dictionary/english/subject-matter-expert>

- 1.4.2. Aggregate: something formed by adding together several amounts or things

Cambridge University Press & Assessment. (n.d.). Aggregate. In *Cambridge dictionary*. Retrieved September 28, 2026, from <https://dictionary.cambridge.org/dictionary/english/aggregate>

- 1.4.3. Dashboard: part of a device, a computer program, etc. that shows information and statistics (= numbers) about how the device, program, etc. is working, which you can use to control it

Cambridge University Press & Assessment. (n.d.). Dashboard. In *Cambridge dictionary*. Retrieved September 28, 2026, from <https://dictionary.cambridge.org/dictionary/english/dashboa>

1.5. References

- 1.5.1. Essential needs and Connection
<https://retrieversupport.umbc.edu/retriever-essential-needs/>
- 1.5.2. About Retriever CARE
<https://retrieversupport.umbc.edu/retriever-care/about/>
- 1.5.3. Retriever Essentials
<https://retrieveressentials.umbc.edu/>

2. Overall Description

2.1. Description

The retriever essential dashboard is designed to assist student help organizations as well as school administration to view the various metrics that each of these organizations track. This will be represented in a centralized hub where each of these stakeholders can view the data collected in a more

digestible view rather than the traditional spreadsheets. By having the ability to view this data in a centralized platform, it allows for metrics to be easily tracked by administration rather than collecting the individual data from different help organizations using different software to collect their own data.

2.2. User classes

User class	Description and Characteristics
Retriever Essentials	The primary audience for this software, made for the ones that need to view their overall trends of the programs that they are involved in. They need to be able to see trends such as the percentage of students that graduated that were previously or currently enrolled in the program. They must be granted access to not only utilize this software but to upload aggregated reports to give more information to the software to effectively view it.
UMBC Administrators/Faculty	The Administrators will be able to only view this dashboard, as the data that will flow through this software will be

	provided by the student support organizations themselves such as retriever essentials using the software they use to track data. As they have no use to be able to upload data, they will not have permission to edit it unless given to by certain directives made by the Software Administrators.
Software Administrators	The software administrators of this service will hold the delegation of accounts to various student bodies and administrators. They will control the overall access to all of the accounts that are made in this software. Additionally, they have the responsibility to ensure that Confidentiality, Integrity, and Availability is consistent through the distribution and development of the software.

2.3. Operating environment

- 2.3.1. The software should be easily accessible through the internet browser, running on any device that has a way to access the web.
- 2.3.2. Will be optimized for desktop service specifically

2.4. Design and Implementation Constraints

- 2.4.1. The software will be able to handle multiple users using the software at once, without a major dip in performance
- 2.4.2. The software will continue to be updated and accurate without failure continuously
- 2.4.3. The software will present a likeness that represents UMBC

2.5. Assumptions

- 2.5.1. The software will be able to represent data given no matter what organization software was used to aggregate the data
- 2.5.2. The software will only be available to a limited number of people, specifically people in high ranking positions in faculty and student support organizations

2.6. Dependencies

- 2.6.1. Data source availability in the form of a csv from stakeholders
- 2.6.2. Cloud infrastructure from AWS to maintain availability of the program
- 2.6.3. FastAPI framework built in doc interface

3. Business Rules

Rule ID	Rule Definition	Type	Impacted Requirements
Batch Upload	Data submissions through a set interval by departments. Does not support real-time syncing.	Constraint	4.1.2
Upload Reminder	Automated notification should be sent from system to uploaders when submission is due.	Implementation	4.1.7
Pre-Department Mapping	Departments use different software and column formats. So fields for each department must be uploaded individually instead of a shared format.	Constraint	4.1.2, 4.1.3
Manual Entry Option	Staff should be able to enter data manually for services	Implementation	4.1.4

	that are not tracked in any existing software or spreadsheet.		
Upload Correction	Uploading incorrect data has to be able to be deleted and re-upload the correct version.	Implementation	4.1.5, 4.1.6
Role-Based Access	Users being able to see or do something depends on the assigned role such as System Admin, Leadership, or Department Contributor.	Constraint	4.1.8,4,1.6
Upload Authorization	Only admins and department staff could upload data while students and public only have view access to what leadership	Constraint	4.1.2

	authorizes.		
Data Sensitivity	Not sure what submitted data will be sensitive or how it will be flagged.	Constraint	NA

4. Functional Requirements

4.1. Department Contributors (Staff)

4.1.1. Users have to to log in using a valid username and password stored in the system before they can access any data or upload features

4.1.2 Users could upload a batch file that contains the service data of the department for a reporting period

4.1.3. The system should validate uploaded files against the department's wanted format and then display errors found such as missing fields.

4.1.4. Users have the ability to manually enter data points as a reported total for services that are not tracked in existing software.

4.1.5. Users should have the ability to delete data that was uploaded as an error

4.1.6. Users should have the ability to re-upload data that was corrected after deleting

4.1.7. Users should receive an automated notification as a reminder for when the next scheduled upload is due.

4.1.8. Users should see the data and visualizations of their own department after logging in.

4.2. Leadership (Dean of Students, Chief of Staff)

4.2.1 Should be able to view a divisionwide dashboard that shows data from all participating departments.

4.2.2 Should be able to filter and compare the metrics by department, program, and time period.

4.2.3 Should be able to generate a summarized selected metrics report

4.2.4 Should be able to view both qualitative and quantitative student stories and metrics on the dashboard.

4.2.5 Should be able to manage user accounts and roles as a secondary capability alongside the system admin.

4.3. System Administrators

4.3.1. Should be able to create, modify, and remove user accounts

4.3.2. Should be able to assign and change the roles and permissions of users.

4.3.3 Should be able to add, edit, or remove participating departments or organizations from the system.

5. Use Cases

5.1. Use Case 1

Name: Log in

Requirement ID: 4.1.1 Users have to log in using a valid username and password stored in the system before they can access any data or can upload anything

Actors: Department contributors, University Leadership, System administrators

Secondary Actors: backend, database

Pre-conditions: The application is running and the user has an account that is in the system

Trigger: The user tries to log in

Post-conditions: The user is logged in and can access the features allowed in their role

Main Success Path: The user enters their username and password into the login page. The system checks the information against the user account stored in the database. If the login information is correct, the system determines the role assigned to the account and gives the user access to the features allowed for that role

Actor Actions:

1. User opens the application. The system displays the login page.
2. User enters their username and password. The system receives the login information.
3. User selects the login button. The system checks the username and password against the database

4. User provides valid login information. The system identifies the users account and assigned role.
5. User enters the application. The system displays the dashboard and features the user has permission to access.

Alternate Flow: If the username or password is incorrect the system denies the access and tells that to the user

5.2. Use Case 2

Name: import department data batch

Requirement ID:

4.1.2 Users could upload a batch file that contains the service data of the department for a reporting period

4.1.3 The system should validate uploaded files against the departments format and display errors like missing fields

4.1.5 Users should be able to delete data that was uploaded as an error

4.1.6 Users should be able to re-upload data that was corrected after deleting

Actors: Department contributors, system administration

Secondary Actors: external data source, backend

Pre-conditions: the user is logged in and has permission to upload the data. The department has a CSV file available to upload

Trigger: The user selects the option to upload department data

Post-conditions: if the uploaded data is valid the system creates an import batch and the data can be used in the dashboard

Main Success Path: The user selects department data file and uploads it into the system. The system checks if the file is in the right format. If the file is, the

system saves the import information and loads the data so it can be used in the dashboard.

Actor Actions:

1. User selects the import data batch option. The system displays the data upload page.
2. User selects the department data file. The system receives the selected file.
3. User submits the file. The system checks the file against the departments required format.
4. User waits for validation. The system checks for problems like missing fields or incorrect formatting.
5. User completes a successful upload. The system makes an import batch record and loads the data.
6. User returns to the dashboard. The system makes new data available to the users who have permission to use it.

Alternate Flow: If the file that is uploaded does not match the format, the system does not accept it and displays an error. If incorrect data was already uploaded a user that has access can delete it and upload the correct version.

5.3. Use Case 3

Name: view and filter dashboard metrics

Requirement ID:

4.1.8 Users should see the data and visualization of their own department after logging in

4.2.1 Leadership should be able to view a division wide dashboard that shows data from all departments

4.2.2 Leadership should be able to filter and compare the metrics by department, program, and time period

4.2.4 Leadership should be able to view qualitative and quantitative student stores and metrics on the dashboard

Actors: Department contributors, University leadership, System administrators

Secondary Actors: backend, database

Pre-conditions: The user is logged in and there is data in the system

Trigger: The user opens the dashboard and selects dashboard metrics, or a filter

Post-conditions: The dashboard displays the requested information based on the users role and selected filters.

Main Success Path: The user opens the dashboard and chooses the information they want to view. The system checks if the user has permission to see it and then displays the metrics. The user can also filter the data by department, program, or time period.

Actor Actions:

1. User opens the dashboard. The system checks the users roles and checks what information they can view.
2. User selects Dashboard metric. The system retrieves the correct data.
3. User selected a department, program, or time filter. The system applies the selected filter
4. User confirms the filters. The system updates the dashboard with the filtered results.
5. User reviews the results. The system displays the data until the filters are changed

5.4. Use Case 4

Name: Export / Download report

Requirement ID: 4.2.3 Leadership should be able to generate a summarized selected metrics report.

Actors: University Leadership and other users with permission to generate reports

Secondary Actors: Backend, database

Pre-conditions: The user is logged in, has permission to generate a report, and there is dashboard data available.

Trigger: The user selects the export / download report option

Post-conditions: The system creates a summary report using the selected data and makes it available to the user

Main Success Path: The user selects the dashboard metrics they want to include in the report and applies any filters they want. The user selects the export / download option. The system collects the selected data and creates a summarized report.

Actor Actions:

1. User selects the metrics to include. The system records the selected metrics
2. User applies filters such as the department or time period. The system retrieves the matching data.

3. User selects export / download report. The system creates a summarized report.
4. User requests the completed report. The system makes the report available for download

Alternate Flow: If there is no data available for the filters, the system tells the user that there is no data for the report.

5.5. Use Case 5

Name: Manage staff Users and roles

Requirement ID:

4.3.1 system administrators should be able to create, modify, and remove user accounts

4.3.2 system administrators should be able to assign and change roles and permissions of users

Actors: system administrators

Secondary Actors: backend, database

Pre-conditions: The user is logged in and has permission to manage user accounts

Trigger: the user selects the manage staff users option.

Post-conditions: The selected account or role is created, changed, or removed and the updated information is saved by the system

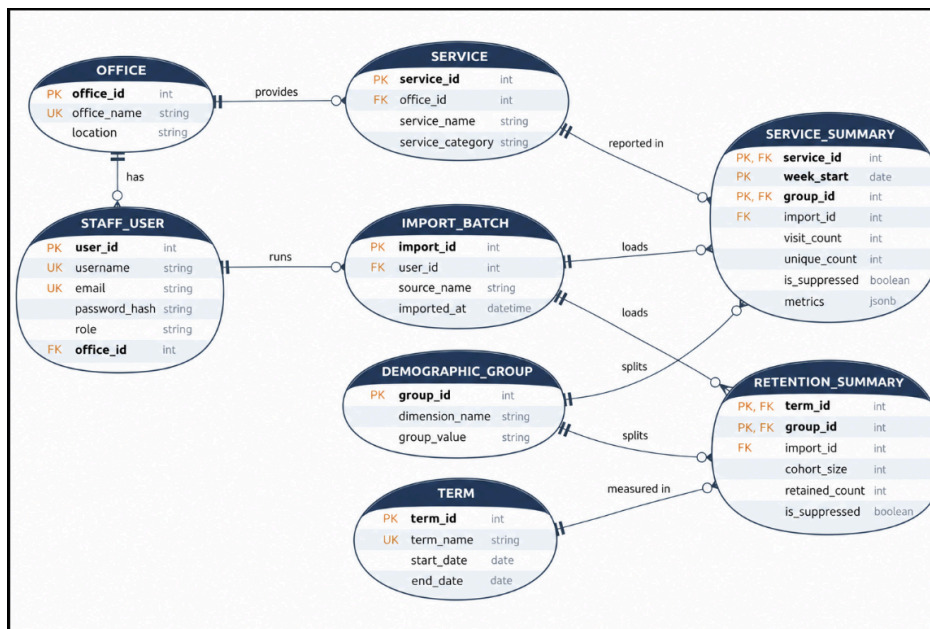
Main Success Path: The system admin opens the user management section and either selects an account or creates a new one. The administrator enters or changes the users information, assigns them a role and permissions, and saves the changes. The system checks the information and updates the account.

Actor Actions:

1. Administrator opens manage staff users. The system displays the user accounts the administrator can manage.
2. Administrator selects an existing account or chooses to create a new account. The system displays the account options
3. Administrator enters or changes account information. The system checks the information.
4. Administrator assigns or changes the user's role and permissions. The system updates the role and permissions for that account.
5. Administrator saves the changes. The system updates the account and confirms that the changes were made.
6. Administrator chooses to remove an account if needed. The system removes the selected account.

Alternate Flow: If the account information is incomplete or invalid, the system does not save the changes and tells the administrator what needs to be fixed.

6. Data Requirements



6.1. Entity - Relationship Diagram

The Entity- Relationship diagram shows the current data model for the retriever essentials dashboard. The main entries are OFFICE, SERVICE, STAFF_USER, IMPORT_BATCH, DEMOGRAPHIC_GROUP, TERM, SERVICE_SUMMARY, and RETENTION SUMMARY.

OFFICE provides SERVICE:

An office can provide different services. Each service is connected to an office through the office_id foreign key

OFFICE has STAFF_USER:

A Department contributor is connected to the office or department they belong to through office_id. This allows the system to determine which departments data the user is allowed to access.

STAFF_USER runs IMPORT_BATCH:

A staff user can upload department data. Each import batch keeps track of the user who performed the upload through user_id.

SERVICE is reported in SERVICE_SUMMARY:

The SERVICE_SUMMARY table stores aggregate information related to a specific service.

IMPORT_BATCH loads SERVICE_SUMMARY:

Each service summary can be connected to the import batch that added the data to the system.

IMPORT_BATCH loads RETENTION_SUMMARY:

Retention summary data can also be connected to the import batch that added it to the system.

DEMOGRAPHIC_GROUP splits SERVICE_SUMMARY:

Service information can be separated into different demographic groups

DEMOGRAPHIC_GROUP splits RETENTION_SUMMARY:

Retention information can also be separated based on demographic groups

TERM is measured in RETENTION_SUMMARY:

Retention information is connected to a specific academic term

6.2. 6.2.1. OFFICE

office_id

Type: int

Constraint: Primary Key

Description: A unique ID for each participating office.

office_name

Type: string

Constraint: Unique Key

Description: The name of the participating office.

location

Type: string

Constraint: None

Description: The location of the office.

6.2.2. SERVICE

service_id

Type: int

Constraint: Primary Key

Description: A unique ID for each service.

office_id

Type: int

Constraint: Foreign Key referencing OFFICE.office_id

Description: Shows which office provides the service.

service_name

Type: string

Constraint: None

Description: The name of the service.

service_category

Type: string

Constraint: None

Description: The category that the service belongs to.

6.2.3. STAFF_USER

user_id

Type: int

Constraint: Primary Key

Description: A unique ID for each staff user.

username

Type: string

Constraint: Unique Key

Description: The username that the user enters to login

password_hash

Type: string

Constraint: Required

Description: Stores the users hashed password so that the actual password is not stored directly in the database

email

Type: string

constraint : Unique Key

Description : The email connected to the user account

role

Type: string

Constraint: None

Description: Stores the user's assigned role for role-based access.

Office_id

Type:int

Constraint: Foreign key referencing OFFICE.office_id

Description: shows which office or department the staff user belongs to. This field can be left empty for university leadership and system admin who aren't limited to one department

6.2.4. IMPORT_BATCH

import_id

Type: int

Constraint: Primary Key

Description: A unique ID for each data upload.

user_id

Type: int

Constraint: Foreign Key referencing STAFF_USER.user_id

Description: Shows which staff user performed the upload.

source_name

Type: string

Constraint: None

Description: Stores the source of the uploaded data.

imported_at

Type: datetime

Constraint: None

Description: Stores the date and time when the data was uploaded.

6.2.5. DEMOGRAPHIC_GROUP

group_id

Type: int

Constraint: Primary Key

Description: A unique ID for each demographic group.

dimension_name

Type: string

Constraint: None

Description: The type of demographic category being used.

group_value

Type: string

Constraint: None

Description: The value that belongs to that demographic category.

6.2.6. TERM

term_id

Type: int

Constraint: Primary Key

Description: A unique ID for an academic term.

term_name

Type: string

Constraint: Unique Key

Description: The name of the academic term.

start_date

Type: date

Constraint: None

Description: The date when the term begins.

end_date

Type: date

Constraint: None

Description: The date when the term ends.

6.2.7. SERVICE_SUMMARY

service_id

Type: int

Constraint: Primary Key and Foreign Key referencing SERVICE.service_id

Description: Shows which service the summary belongs to and is part of the composite primary key.

week_start

Type: date

Constraint: Primary Key

Description: The starting date of the reporting week and is part of the composite primary key.

group_id

Type: int

Constraint: Primary Key and Foreign Key referencing

DEMOGRAPHIC_GROUP.group_id

Description: Shows which demographic group the summary belongs to and is part of the composite primary key.

import_id

Type: int

Constraint: Foreign Key referencing IMPORT_BATCH.import_id

Description: Shows which import batch added the data.

visit_count

Type: int

Constraint: None

Description: The total number of service visits in the summary.

unique_count

Type: int

Constraint: None

Description: The number of unique records included in the summary.

is_suppressed

Type: boolean

Constraint: None

Description: Shows whether the data should be hidden from being displayed.

metrics

Type: jsonb

Constraint: None

Description: Stores additional service metrics.

6.2.8. RETENTION_SUMMARY

term_id

Type: int

Constraint: Primary Key and Foreign Key referencing TERM.term_id

Description: Shows the academic term connected to the retention summary and is part of the composite primary key.

group_id

Type: int

Constraint: Primary Key and Foreign Key referencing

DEMOGRAPHIC_GROUP.group_id

Description: Shows which demographic group the retention summary belongs to and is part of the composite primary key.

import_id

Type: int

Constraint: Foreign Key referencing IMPORT_BATCH.import_id

Description: Shows which import batch added the retention data.

cohort_size

Type: int

Constraint: None

Description: The total size of the group being measured.

retained_count

Type: int

Constraint: None

Description: The number of people in the group who were counted as retained.

is_suppressed

Type: boolean

Constraint: None

Description: Shows whether the retention data should be hidden from being displayed.

6.3. Data Maintenance

6.3.1. Data Import and Validation

Department data will be added to the system through batch uploads instead of real time syncing. Before accepting an uploaded file, the system should check that the data matches the format expected for that department. If fields are missing or incorrectly formatted, the system should show the errors and not accept the file until it is corrected.

6.3.2. Incorrect and Corrected Uploads

Users with the correct permissions should be able to delete data that was uploaded incorrectly and upload a corrected version. Each upload will be connected to an `IMPORT_BATCH` record so the system can keep track of where the data came from, who uploaded it, and when it was uploaded.

6.3.3. Upload History

The application should keep a history of previous uploads. This will allow the system to keep track of earlier data submissions, the user who performed the upload and the date and time when it happened.

6.3.4. Role-Based Data Access

The data that a user can access should depend on their assigned role. Department Contributors should only be able to access the information and features allowed for their department. University Leadership should be able to view approved information across participating departments.

System Administrators should have access to administrative features such as managing users, roles, and participating departments.

6.3.5. Aggregate and Suppressed Data

The current data model stores service and retention information as aggregate data instead of individual student records. Both SERVICE_SUMMARY and RETENTION_SUMMARY include an is_suppressed field, which can be used to show whether certain information should be hidden from the dashboard.

The exact rules for what data should be considered sensitive and when data should be suppressed have not been fully decided yet. These rules will need to be clarified with the stakeholders later in the project.

6.3.6. Qualitative Student Stories

The current functional requirements state that University Leadership should be able to view both qualitative and quantitative student stories and metrics. However, the current Sprint 1 ER Diagram does not include a separate table for qualitative student stories. Because of this, the way student stories will be stored still needs to be discussed and decided with the stakeholders in a later version of the SRS.

7. External Interface Requirements

7.1. User Interface

- 7.1.1. The application must accept keyboard and mouse inputs.
- 7.1.2. The application should have an intuitive interface and layout so as to not confuse users.

7.2. Software Interface

- 7.2.1. The application should allow for data to be queried.
- 7.2.2. Data should be allowed from the accepted formats.
- 7.2.3. Graphs should accurately display data to the user.

7.3. Communication Interface

- 7.3.1. Reminders should be sent out via email to upload data.

8. Nonfunctional Requirements

8.1. Availability Requirements

- 8.1.1. The application must work every time.
- 8.1.2. The application should not crash.

8.2. Reliability Requirements

- 8.2.1. Data upload rate must be fast.
- 8.2.2. Data querying speed must be quick.

8.3. Scalability Requirements

- 8.3.1. Graphs should allow for large date ranges.
- 8.3.2. Upload history should be kept.

8.4. Security Requirements

- 8.4.1. Confidentiality and private data must be kept properly.

9. Reporting

9.1. Report 1

- 9.1.1. Should clearly explain the current state of the project.
- 9.1.2. Should clearly explain what was learned from the stakeholders and the process.
- 9.1.3. Shows the scope of the following Sprint.
- 9.1.4. Gives Sprint 1 burndown chart.

10. Supplemental Info

10.1. Frontend

For the frontend we will be using React to process the visual data that comes from the stakeholders, the buttons to upload the csv to process the data, as well as to display the login page. It will also be responsible for the visualization of handling the user interaction. It will communicate with the backend exclusively with FastAPI's REST endpoints (JSON over HTTPS).

10.2. Backend

The backend will be purely made through FastAPI and this is the core logic layer. It is responsible for communicating with the front end to ensure the user buttons are used as designed. It handles the receiving and parsing of uploaded CSV files from each department. Running validation checks of the data that was uploaded.

Enforcing authentication and authorization for the platforms to control why the users can and cannot interact with.

10.3. Coding practices

All code is managed through git and uses GitHub as the UI to view the git repositories. Docker and docker compose will be run on every system during development to ensure that every user has the same environment to avoid conflict when merging into the codebase. FastAPI's built in /docs endpoint will be used for fast testing when needing to test the backend logic. The data validation of the pydantic library ensures that all data receives a type check before being normalized. Credentials, API keys, and SSH keys will be in a secrets manager and never hardcoded. Environment variables will never be pushed to any repository under any circumstance.